



Неделя 9: RAG как часть FastAPI

О чём эта неделя

На прошлой неделе вы дали проекту бэкенд: данные в Postgres, FastAPI с `/users`, админка. Но `vildly-rag-bot` называется rag-ботом не просто так — пора добавить его главную способность. Мы сделаем из него **репетитора по учебнику Коммервилла «Software Engineering»** — той самой книги, что идёт через весь курс в разделах «Что почитать». Студент спрашивает «чем верификация отличается от валидации», а бот отвечает **по книге**, а не из общей памяти модели.

Это и есть RAG (retrieval-augmented generation): бот сначала **находит** в книге нужные фрагменты, а потом **отвечает** на их основе — со ссылкой на то, откуда взят ответ. Собирать будем по той же логике, что и на Неделе 8: RAG прячется за сервис, а сверху к нему подключаются два входа — эндпоинт `/ask` в FastAPI (рядом с `/users`) и команда `/ask` в боте. Один движок знаний, два потребителя.

Что предстоит сделать

- завести базу знаний на **pgvector** (в том же Postgres) и загрузить в неё учебник Коммервилла;
- собрать RAG-агента на **agno** и спрятать его за обёртку `RagService`;
- отдать RAG наружу эндпоинтом `POST /ask` в FastAPI;
- подключить к тому же `RagService` команду `/ask` в боте;
- убрать ключ LLM в секреты (`.env` / GitHub Secrets), как токен бота на прошлых неделях.



Сквозной пример этой недели

`vildly-rag-bot`. К концу недели: в `http://localhost:8000/docs` появляется `POST /ask` — задаёте вопрос по учебнику Соммервилла («какие бывают уровни тестирования?») и получаете ответ со ссылкой на главу; та же команда `/ask` работает в Telegram. И веб, и бот ходят в один `RagService` и одну базу `pgvector` с проиндексированной книгой.



Зачем RAG, если можно просто спросить у LLM?

Голая модель отвечает «в среднем по интернету» и легко выдумывает то, чего не знает — а на экзамене вам нужен ответ именно по Соммервиллу, а не «примерно похоже». RAG сначала достаёт из **книги** релевантные фрагменты и кладёт их модели в контекст. Ответ становится основанным на источнике, и вы можете проверить, из какой главы он взят.

Часть 1. Книга в базе знаний на `pgvector`

RAG держится на **векторном поиске**: каждый кусок текста превращается в вектор (эмбединг), и на вопрос находятся куски с самыми близкими векторами. Хранить их будем в **`pgvector`** — расширении Postgres, которое у вас уже стоит в образе базы с Недели 8. Отдельная инфраструктура не нужна: вектора книги живут в том же Postgres, рядом с пользователями.

Источник знаний — учебник

Источник один и чистый: PDF учебника Соммервилла (ваш экземпляр для учёбы). Одна хорошая книга — идеальный первый корпус для RAG: связный текст, понятная структура глав.

```
# bot/rag/knowledge.py
from agno.knowledge.pdf import PDFKnowledgeBase      # имя класса сверьте с доками agno
from agno.vectordb.pgvector import PgVector
```

```

knowledge = PDFKnowledgeBase(
    path="knowledge/sommerville",           # папка с PDF уче
бника
    vector_db=PgVector(
        table_name="knowledge",           # отдельная табли
ца в том же Postgres
        db_url=KB_DATABASE_URL,           # см. врезку про
драйвер ниже
    ),
)

```

Загрузка (индексация)

Книгу нужно один раз проиндексировать — разбить на куски, посчитать эмбединги, сложить в pgvector. Делаем это отдельной командой, а не при каждом старте бота: учебник большой, гонять индексацию на каждый запуск незачем.

```

# scripts/load_knowledge.py
import asyncio
from bot.rag.knowledge import knowledge

async def main():
    await knowledge.aload(recreate=False) # recreate=True –
если заменили файл книги
    print("Учебник проиндексирован")

if __name__ == "__main__":
    asyncio.run(main())

```

```
python -m scripts.load_knowledge
```



Почему у базы знаний своя строка подключения?

На Неделе 8 SQLAlchemy подключался драйвером `asyncpg` (`postgresql+asyncpg://...`). `agno` для `pgvector` обычно ждёт драйвер `psycopg` (`postgresql+psycopg://...`). База одна и та же — отличается только драйвер в строке. Держите две константы (`DATABASE_URL` и `KB_DATABASE_URL`) на один Postgres и не путайте их.



Совет наставника

Индексация большого PDF идёт не мгновенно и тратит запросы к эмбединг-модели (тоже платные). Прогоняйте `load_knowledge` один раз и только при смене файла книги — не вставляйте её в старт бота. Захотите позже добавить вторую книгу или методички курса — просто положите файлы в ту же папку и перезапустите загрузку.

Часть 2. Агент на `agno` за обёрткой `RagService`

Теперь сам RAG. `agno`-агент с базой знаний сам делает поиск по книге и зовёт LLM. Но, как и с базой на Неделе 8, наружу мы выставляем не фреймворк, а **свой** тонкий `RagService` — тогда и API, и бот зовут одно и то же, а на будущей неделе с тестами этот шов легко замочать.

```
# bot/rag/agent.py
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from bot.rag.knowledge import knowledge

def build_agent():
    return Agent(
        model=OpenAIChat(id="gpt-4o-mini"), # дешёвая модель
        knowledge=knowledge,
        search_knowledge=True, # агент сам ищет по книге перед ответом
        instructions="Отвечай только по учебнику. Если в нём"
```

```
нет ответа — так и скажи.",
    markdown=True,
)
```

```
# bot/rag/service.py
class RagService:
    def __init__(self, agent): # agno Agent в
# проде, фейк в тестах
        self._agent = agent

    async def answer(self, question: str) -> str:
        question = question.strip()
        if not question:
            return "Задайте вопрос после команды, например: /
ask что такое рефакторинг"

        result = await self._agent.arun(question) # тут agn
о ищет по книге и зовёт LLM
        text = (result.content or "").strip()
        return text or "Не нашёл ответа в учебнике. Попробуйт
е переформулировать."
```



Правило простоты: один RagService — два входа

Не пишите логику ответа дважды — в боте и в API. Она живёт в `RagService`, а бот и FastAPI — просто два потребителя, ровно как один `UserRepository` обслуживал бота, API и админку на Неделе 8. Захотите завтра сделать веб-страницу «спроси по учебнику» — подключите третий вход к тому же сервису.



Совет наставника

Обратите внимание на `instructions`: мы прямо просим агента отвечать только по книге и честно признаваться, когда ответа в ней нет. Без этой инструкции модель охотно «дополнит» ответ из общей

памяти — а вам нужен именноommerвилл, иначе теряется весь смысл RAG.

Часть 3. Эндпоинт /ask и бот на одном сервисе

/ask в FastAPI

Добавляем POST-эндпоинт рядом с `/users`. Вход и выход описываем pydantic-моделями — FastAPI сам их провалидирует и покажет в `/docs`.

```
# api/main.py (добавляем к прошлой неделе)
from pydantic import BaseModel
from fastapi import Depends
from bot.rag.agent import build_agent
from bot.rag.service import RagService

rag_service = RagService(build_agent()) # собираем один
раз на старте

def get_rag() -> RagService:
    return rag_service

class AskRequest(BaseModel):
    question: str

class AskResponse(BaseModel):
    answer: str

@app.post("/ask", response_model=AskResponse)
async def ask(body: AskRequest, rag: RagService = Depends(get_rag)):
    return AskResponse(answer=await rag.answer(body.question))
```

Откройте `/docs` → `POST /ask` → «Try it out», спросите что-нибудь из книги («чем отличается инкрементная разработка от водопадной?») — увидите ответ, собранный из учебника.

/ask в боте — через тот же сервис

Команда бота ничего не знает про `agno`: она берёт текст вопроса и отдаёт его `RagService`.

```
# bot/handlers/ask.py
from aiogram import Router
from aiogram.filters import Command
from aiogram.types import Message

router = Router()

@router.message(Command("ask"))
async def ask_handler(message: Message, rag: RagService):
    question = message.text.removeprefix("/ask")
    await message.answer(await rag.answer(question))
```

Ключ LLM — в секреты

Агент ходит в платную модель, значит появляется ключ (`OPENAI_API_KEY` или ключ вашего провайдера). Он живёт там же, где `BOT_TOKEN`: в `.env` локально и в GitHub Secrets для деплоя — **никогда в git**.

```
# compose.yaml — добавляем ключ сервисам, которые отвечают
api:
  environment:
    DATABASE_URL: postgresql+asyncpg://bot:${DB_PASSWORD}@db:5432/bot
    KB_DATABASE_URL: postgresql+psycpg://bot:${DB_PASSWORD}@db:5432/bot
    OPENAI_API_KEY: ${OPENAI_API_KEY}
```



Совет наставника

Не хотите платить за облачную модель на время учёбы — agno умеет работать с локальными моделями (например, через Ollama). Интерфейс `RagService` от этого не меняется: меняете только строку с моделью в `build_agent()`, всё остальное — база знаний, эндпоинт, бот — остаётся как есть. Это та же выгода от обёртки, что и со сменой БД.



А качество ответов — как проверить?

На глаз этой недели достаточно: задали пару вопросов, ответы на которые точно знаете по книге, и сверили. Строгая оценка RAG (релевантность, галлюцинации, полнота) — большая отдельная тема, и это не то, на чём стоит застревать сейчас. На следующей неделе мы займёмся **тестами**: как проверять `/ask` и `RagService` автоматически, замокав LLM, чтобы тесты были быстрыми и бесплатными.

Задачи на эту неделю

Отмечайте по мере готовности.

1. Книга в базе знаний

- ☐ PDF учебника Соммервилла в папке `knowledge/sommerville`
- ☐ `bot/rag/knowledge.py`: knowledge base на `PgVector` (таблица `knowledge`)
- ☐ `KB_DATABASE_URL` (драйвер `psycopg`) в `.env`
- ☐ `scripts/load_knowledge.py` индексирует книгу без ошибок

2. Агент и сервис

- ☐ `bot/rag/agent.py`: agno-агент с `knowledge`, `search_knowledge=True` и инструкцией «только по книге»

- ☐ `bot/rag/service.py` : `RagService` с внедряемым агентом и фолбэками

3. Два входа

- ☐ `POST /ask` в FastAPI с pydantic-моделями, виден в `/docs`
- ☐ `/ask` в боте отвечает через тот же `RagService`
- ☐ `OPENAI_API_KEY` (или ключ провайдера) в `.env` и в GitHub Secrets, не в git

4. Ритм недели

- ☐ Issue → branch → PR → merge на фичу RAG (см. issues в репозитории)
- ☐ В пятницу: рефлексия итерации в `docs/iterations/`, одной фразой — план следующей недели (тестирование)

FAQ: частые вопросы

Чем `/ask` через API лучше, чем просто в боте?

API — переиспользуемый вход: завтра к нему подключится веб-страница «спроси по учебнику», мобильный клиент или другой сервис, и логику отвечать не придётся писать заново. Бот при этом остаётся одним из клиентов. Один `RagService` — сколько угодно входов.

Где лежит книга — нужна ли отдельная база?

Нет. `pgvector` — это расширение того же Postgres, что и на Неделе 8; вектора глав книги хранятся в отдельной таблице рядом с пользователями. Никакой новой инфраструктуры — одна база на всё.

RAG — это дообучение модели на книге?

Нет. Вы не трогаете саму модель. Вы индексируете книгу в базу знаний, а на каждый вопрос агент достаёт из неё релевантные куски и подаёт модели как контекст. Заменяли издание учебника — просто перезагрузили индекс, без всякого обучения.

Можно ли добавить не только Соммервилла?

Да. Положите в ту же папку другие материалы курса (методички, статьи) и перезапустите индексацию. Но качество источников важнее количества: одна

аккуратная книга лучше, чем куча случайных PDF вперемешку.

Обязательно ли платная модель?

Нет. Для учёбы возьмите дешёвую модель или локальную через Ollama — `RagService` и всё вокруг не меняются, меняется одна строка в `build_agent()`. Но помните: и вызовы модели, и подсчёт эмбеддингов при индексации платные, ключ держите в секретах.

Почему у базы знаний строка подключения с другим драйвером?

База одна (Postgres), но SQLAlchemy на Неделе 8 использует `asyncpg`, а agno для pgvector — `psycopg`. Отсюда две строки на один и тот же Postgres: `DATABASE_URL` и `KB_DATABASE_URL`. Это нормально, просто не перепутайте их местами.

Что почитать дополнительно

- Ian Sommerville, «Software Engineering» — та самая книга, по которой теперь отвечает ваш бот. software-engineering-book.com
- Документация agno: knowledge base, vector DB (pgvector), агенты — docs.agno.com
- FastAPI, тело запроса и pydantic-модели: fastapi.tiangolo.com/tutorial/body
- Что такое RAG простыми словами (Яндекс Практикум / Хабр): practicum.yandex.ru/blog
- pgvector — векторный поиск в Postgres: github.com/pgvector/pgvector



Застряли с индексацией книги или подключением agno к pgvector больше часа — не бейтесь в одиночку. `could not connect`, несовпадение драйвера (`asyncpg` vs `psycopg`), пустая выдача поиска — типовые вещи. Скопируйте команду, лог и вопрос — и спросите наставника или сокурсника.